🐾 YourLover 桌面 UI 开发指南

> ✨ 本文档说明桌面端（悬浮宠物 + 聊天主页 + 设置面板）的整体架构、事件总线机制、各组件/窗口职责，以及如何在此基础上扩展新功能。

📐 1. 整体架构概览

桌面 UI 采用 **组件化 + 事件总线** 架构，目标是把窗口、控件、业务逻辑彻底解耦：

```
windows/            → 所有顶层窗口（悬浮窗、聊天窗、设置窗）
widgets/            → 可复用组件（聊天组件、右键菜单、设置项等）
core/event_bus.py   → 全局事件总线，所有跨模块通信都通过它
demos/desktop_ui_demo.py → 可运行的演示入口，用假 Avatar 验证 UI 流程
```

🔑 1.1 核心设计原则

原则	说明
🧩 **窗口解耦**	窗口之间不直接互相调用。`LoverWindow` 不会直接 `chat_window.show()`，而是发射 EventBus 信号。
🎯 **组件内聚**	组件只负责自身 UI 与内部信号。例如 `ChatInput` 只会在用户点击发送时发射 `send_clicked(text)`，不直接调用 Agent。
🕹️ **统一协调**	`WindowManager` 是唯一的窗口协调者，订阅 EventBus，把事件分发给对应窗口。
🐰 **Avatar 解耦**	`LoverWindow` 只负责容器和原始鼠标事件，具体 Live2D 渲染通过 `AvatarControllerProtocol` 解耦。

📡 2. 事件总线（EventBus）机制

2.1 什么是 EventBus

`core/event_bus.py` 中定义了一个全局单例 `event_bus`，继承自 `QObject`，内部使用 PySide6 的 `Signal` 定义所有跨模块事件。

任何模块都可以通过以下方式通信：

```python
from core.event_bus import event_bus

# 发送事件
event_bus.show_chat_window_requested.emit()

# 接收事件
event_bus.show_chat_window_requested.connect(on_show_chat)
```

2.2 信号传递流程

一个典型的事件链路如下：

```
用户操作（点击/输入/拖拽）
        ↓
具体组件（MenuItem / ChatInput / LoverWindow）
        ↓
EventBus 信号发射
        ↓
WindowManager 或其他监听者接收并执行动作
```

🚪 举例：点击右键菜单的「退出」

```
用户点击"退出"
    ↓
QAction.triggered
    ↓
MenuItem.on_triggered()
    ↓
event_bus.application_quit_requested.emit()
    ↓
WindowManager._setup_event_routing() 中已连接：
    event_bus.application_quit_requested.connect(self.quit)
    ↓
WindowManager.quit() → self._app.quit()
```

> ⚠️ **注意**：没有直接调用 `sys.exit()`。`WindowManager.quit()` 调用的是 `QApplication.quit()`，这样 Qt 能正常清理资源、关闭窗口、释放对象。

2.3 典型事件流程示例

🏠 示例 A：右键点击「主页」

```
LoverWindow 右键菜单 → MenuItem(HOME).on_triggered()
    → event_bus.menu_home_requested.emit()
    → event_bus.show_chat_window_requested.emit()
        → WindowManager.show_chat() 显示 ChatWindow
```

💬 示例 B：用户发送消息

```
ChatInput._on_send()
    → send_clicked.emit(text)
    → ChatWindow.connect_signals() 中：
        self._input.send_clicked.connect(event_bus.user_message_sent.emit)
    → event_bus.user_message_sent.emit(text)
        → ChatWindow.append_user_message(text) 显示到历史
        → desktop_ui_demo.on_user_message(text) 启动模拟 Agent 回复线程
```

⚡ 示例 C：Agent 流式回复

```
FakeAgentWorker.run()
    → event_bus.agent_thinking_started.emit()
    → event_bus.agent_stream_chunk.emit(chunk)
    → event_bus.agent_reply_received.emit(event)
        → ChatWindow._on_agent_thinking_started()
        → ChatWindow._on_agent_stream_chunk()
        → ChatWindow._on_agent_reply_received()
        → WindowManager._on_agent_reply()（窗口管理层补充逻辑：震动宠物）
```

🪐 示例 D：迷你聊天（快速对话）

迷你聊天通过 `main.py` 中的 `AgentWorker` 调用 `AgentController`，与大聊天窗口共用同一个 Agent，但使用**独立的 session_id** 实现会话隔离。

```
用户点击💬按钮
    → MiniChatOverlay.expand() 展开输入框
    → 用户输入文本 + Enter / 点击➤
    → event_bus.mini_chat_input_sent.emit(text)
        → main.py: on_mini_chat_message(text)
            → 创建 AgentWorker(mini_chat=True, configurable={session_id:"mini-
chat"})
            → AgentWorker.run():
                → event_bus.mini_chat_thinking_started.emit()
                    → MiniChatOverlay.show_thinking()
                    → SpeechBubble.show_thinking() 头顶显示"💭 思考中..."
                    → WindowManager._on_mini_chat_thinking() → AvatarController
播放思考表情
                → AgentController.chat_test(text, configurable)
                → reply_ready.emit(AgentMessage)
                    → main.py: _on_mini_chat_reply(agent_message)
                        → event_bus.mini_chat_reply_received.emit(reply_text)
                            → MiniChatOverlay.show_reply(reply)
                            → SpeechBubble.show_message(reply) 头顶显示回复
                            → WindowManager._on_mini_chat_reply() →
AvatarController 播放说话动作
                            → 8秒后自动隐藏
```

> ✨ **系统提示自动触发**：AgentController 启动时 `APIConnect()` 返回 `AgentMessage("连接成功了！", "Happy")`，`main.py` 将其保存到 `_startup_message`，在 `manager.run()` 后延迟 800ms 通过 `mini_chat_reply_received` 自动显示在角色头顶气泡。

迷你聊天 vs 大聊天窗口的区别：

特性	大聊天窗口	迷你聊天
打开方式	右键"主页"	角色右下角 💬 按钮
消息记录	保存在 `ChatWindow` 历史	显示在头顶气泡，不持久化
主要信号	`user_message_sent` / `agent_reply_received`	`mini_chat_input_sent` / `mini_chat_reply_received`
session_id	每个会话独立 UUID	固定 `"mini-chat"`

特性	大聊天窗口	迷你聊天
思考信号	`agent_thinking_started/finished`	`mini_chat_thinking_started`，`agent_thinking_finished` 不发射

2.4 当前 EventBus 信号分类

分类	信号	说明
🐾 **Avatar 交互**	`avatar_clicked(x, y)`	Avatar 区域被点击
	`avatar_hit_area_clicked(area_name)`	命中 HitArea
	`avatar_drag_started(direction)`	拖拽开始，带初始方向 `"left"/"right"/"up"/"down"`
	`avatar_dragged(dx, dy)`	拖拽中
	`avatar_drag_direction(direction)`	拖拽过程中方向变化
	`avatar_drag_finished(x, y)`	拖拽结束
	`avatar_shaken()`	检测到摇晃
	`avatar_state_changed(state)`	Avatar 状态变化
📋 **菜单/导航**	`menu_home_requested()`	请求主页
	`menu_settings_requested()`	请求设置
	`menu_quit_requested()`	请求退出
	`menu_item_triggered(id, payload)`	通用菜单项触发
💬 **聊天**	`user_message_sent(text)`	用户发送消息
	`chat_session_changed(session_id)`	聊天窗口切换/新建会话，通知 AgentController 更新 session_id
	`agent_reply_received(event)`	Agent 完整回复
	`agent_stream_chunk(chunk)`	流式片段
	`agent_thinking_started/finished()`	Agent 思考状态（仅大聊天模式发射）
💭 **迷你聊天**	`mini_chat_input_sent(text)`	用户在迷你输入框发送消息
	`mini_chat_thinking_started()`	迷你聊天开始思考
	`mini_chat_reply_received(text)`	迷你聊天收到回复（角色头顶气泡显示）
⚙️ **设置**	`setting_changed(key, value)`	单个设置项变化
	`settings_batch_changed(values)`	批量设置变化
	`settings_save_requested()`	请求保存设置

分类	信号	说明
🪟 窗口管理	`show_chat_window_requested()`	显示聊天窗
	`hide_chat_window_requested()`	隐藏聊天窗
	`show_setting_window_requested()`	显示设置窗
	`hide_setting_window_requested()`	隐藏设置窗
	`hide_all_secondary_windows_requested()`	隐藏所有二级窗口
	`application_quit_requested()`	请求退出应用

🧱 3. 组件与窗口职责

🪟 3.1 windows/ 目录

`base_window.py`

所有窗口的抽象基类 `BaseWindow`（继承 `QMainWindow`）。

- 统一窗口生命周期：`show_window()`、`hide_window()`、`toggle()`。
- 子类必须实现 `setup_ui()` 和 `connect_signals()`。
- 默认 `closeEvent` 改为隐藏而非退出。

`lover_window.py`

虚拟伴侣悬浮窗口 `LoverWindow`。

- 无边框、透明、置顶悬浮。
- 承载 Avatar 渲染控件（由外部注入）。
- 处理鼠标拖拽、摇晃检测。
- 右键弹出菜单（主页、设置、退出）。
- 管理 HitArea，把点击事件转发到 EventBus。
- 不负责具体 Live2D 渲染。

关键接口：`set_avatar_widget(widget)`、`register_hit_area(name, rect)`、`shake_avatar()`、`on_avatar_clicked(pos)`、`position_changed(pos)`

`chat_window.py`

聊天主页窗口 `ChatWindow`（Claude Desk 风格）。

- 左右双栏布局：左侧 `ChatSessionList`（会话列表），右侧 `ChatHeader` + `ChatHistory` + `ChatInput`。
- 把用户输入转发到 `user_message_sent`。
- 订阅 Agent 事件并更新历史记录。
- 管理会话：新建、切换、删除，切换时发射 `chat_session_changed(session_id)`。
- 消息追加自动保存到当前会话。

关键接口：`append_user_message(text)`、`append_agent_message(text)`、`update_last_agent_message(text)`、`set_status(text, color)`、`create_new_session()`、`switch_session(session_id)` 等。

`setting_window.py`

设置面板窗口 `SettingWindow`。

- 无边框、可从右侧滑入滑出。
- 动态管理设置项，保存时广播 `settings_batch_changed` 和 `settings_save_requested`。

关键接口：`add_setting_item(item)`、`get_all_values()`、`save_settings()`、`slide_in(anchor)`、`slide_out()`。

`window_manager.py`

窗口管理器。

- 创建并持有 `LoverWindow`、`ChatWindow`、`SettingWindow`、`MiniChatOverlay`。
- 订阅 EventBus，把事件路由到对应窗口。
- 注入 AvatarController 及其渲染控件。
- 管理窗口联动。

关键接口：`run()`、`quit()`、`set_avatar_controller(controller)`、`show_chat()`、`show_settings()`、`mini_chat_overlay`

`mini_chat_overlay.py`

迷你聊天浮层 `MiniChatOverlay`（快速对话入口）。

- 无边框、透明、置顶浮动，始终显示 💬 按钮。
- 点击展开输入框，发送消息走 `mini_chat_input_sent`。
- 监听回复信号，在角色头顶显示 `SpeechBubble`。
- 跟随 `LoverWindow.position_changed` 信号移动，并调用 `self.raise_()` 确保不被遮挡。

关键接口：`set_target_window(window)`、`expand()`、`collapse()`、`show_reply(text)`、`show_thinking()`、`hide_bubble()`。

`avatar/avatar_controller_protocol.py`

> ⚠️ 本文件属于 `avatar/` 模块，不在 `windows/` 下。

定义 `AvatarControllerProtocol`，规定 Avatar 控制器必须实现的方法，包括点击、拖拽、摇晃、模型更新、表情动作等，并返回渲染控件。逻辑上属于 avatar 模块，依赖方向无循环依赖。

🧩 3.2 widgets/ 目录

`base/base_widget.py`

所有可复用组件的抽象基类 `BaseWidget`，强制实现 `setup_ui()` 和 `connect_signals()`。

`base/extensible_item.py`

可扩展列表项接口 `ExtensibleItem`，统一右键菜单项、设置项等的行为，封装 `ItemPayload`。

`menu/menu_builder.py`

右键菜单构建器 `MenuBuilder`，链式 API：`add_home()`、`add_settings()`、`add_quit()`、`add_custom()`、`add_separator()`，最终 `build(parent)` 生成 `ContextMenu`。

`menu/menu_item.py`

单个右键菜单项 `MenuItem`，根据 `MenuItemType` 决定触发行为，如 HOME → 发射 `menu_home_requested` + `show_chat_window_requested`。

`menu/context_menu.py`

右键菜单容器 `ContextMenu`，管理一组 `MenuItem`。

`chat/chat_header.py`

聊天窗口顶部栏，显示头像、标题、状态，提供设置按钮。

`chat/chat_session_list.py`

左侧历史会话列表组件，支持新建、切换、删除会话，选中态粉色高亮，hover 显示删除按钮。

`chat/chat_history.py`

聊天历史记录面板，维护 `ChatBubble` 列表，支持流式更新，自动滚动到底部。

`chat/chat_bubble.py`

单条聊天气泡，用户消息粉色实底白字，Agent 消息白色卡片+粉色边框，系统消息淡黄居中。

`chat/chat_input.py`

聊天输入区域，支持 `Ctrl+Enter` 发送，粉色主题边框与按钮。

`mini_chat/speech_bubble.py`

角色头顶对话气泡 `SpeechBubble`，自绘圆角矩形+底部三角尾巴，支持思考动画和自动隐藏。

`settings/setting_panel.py`

设置项列表面板，动态增删，转发 `setting_changed` 信号。

`settings/setting_item.py`

设置项抽象基类与具体实现：`ToggleSettingItem`、`TextSettingItem`、`SliderSettingItem`、`OptionSettingItem`。

🧠 3.3 core/ 目录

- `event_bus.py`：全局事件总线。
- `agent_event.py`：`AgentEvent` 数据类。

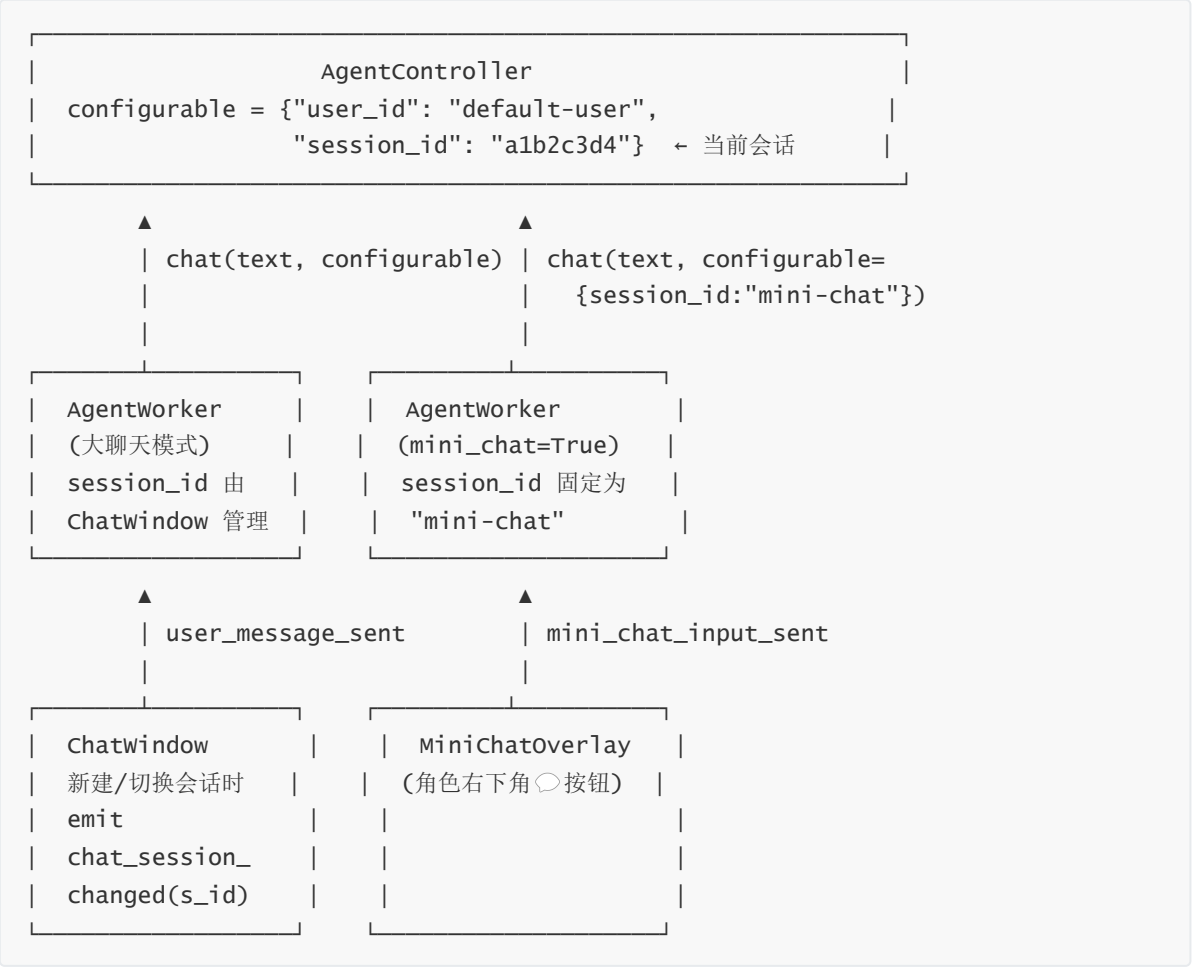
🚀 3.4 main.py（应用主入口）

`main.py` 是桌面助手的真正入口，负责初始化 QApplication、WindowManager、注入 Live2DController、初始化 AgentController、管理大聊天与迷你聊天的会话隔离。

`AgentWorker`（QThread）

封装 `AgentController.chat()` 的同步调用，通过 `mini_chat` 参数控制思考信号路由，`configurable` 参数指定 `session_id`。

会话管理架构



会话隔离规则：

1. 大聊天窗口：每个会话有独立的 UUID `session_id`，切换时通过 `chat_session_changed` 信号通知 `main.py` 更新。
2. 迷你聊天：固定 `session_id="mini-chat"`，在 `AgentWorker` 中传入 `configurable`，不修改全局。
3. 系统提示：`init_agent_controller()` 保存启动消息，延迟 800ms 自动显示在迷你聊天气泡。

`main.py` 中的关键函数速览：

函数	职责
`on_user_message(text)`	大聊天消息处理，创建 AgentWorker，回复走 `agent_reply_received`

函数	职责
`on_mini_chat_message(text)`	迷你聊天消息处理，回复走 `mini_chat_reply_received`
`_on_agent_reply(agent_message)`	包装为 AgentEvent，发射 `agent_reply_received`
`_on_mini_chat_reply(agent_message)`	提取文本，发射 `mini_chat_reply_received`
`on_chat_session_changed(session_id)`	更新 AgentController 的 session_id
`init_agent_controller()`	初始化 AgentController + 连接 LLM，保存系统提示
`main()`	入口：创建所有组件，连接事件，延迟显示系统提示

🛠️ 4. 如何扩展新功能

4.1 新增一个窗口

1. 在 `windows/` 下新建文件，继承 `BaseWindow`，实现 `setup_ui()` 和 `connect_signals()`。
2. 在 `WindowManager.__init__()` 中创建实例。
3. 在 `WindowManager._setup_event_routing()` 中订阅事件。
4. 在 `core/event_bus.py` 中添加新信号。
5. 在触发处发射信号。

4.2 新增一个可复用组件

1. 在 `widgets/` 下新建文件，继承 `BaseWidget`。
2. 实现 `setup_ui()` 和 `connect_signals()`。
3. 在窗口的 `setup_ui()` 中实例化并加入布局。

4.3 新增一个右键菜单项

方案 A：预定义类型（推荐）

1. 在 `MenuItemType` 枚举中新增类型（如 `ABOUT`）。
2. 在 `MenuItem.on_triggered()` 中添加对应的 EventBus 发射逻辑。
3. 在 `MenuBuilder` 中新增 `add_about()` 方法，并在 `build()` 中增加分支。
4. 在 `core/event_bus.py` 中新增信号，并在 `WindowManager` 中连接。

方案 B：自定义项 + callback（局部逻辑）

```
.add_custom("menu_about", "关于", callback=on_about_clicked)
```

4.4 新增一个设置项

1. 如果需要新类型，继承 `SettingItem` 并实现 `create_editor()` 和 `_update_editor()`。
2. 在 `SettingWindow` 中添加实例。
3. 监听 `event_bus.setting_changed` 响应变化。

4.5 新增一个 EventBus 信号

1. 在 `core/event_bus.py` 的 `EventBus` 类中添加 `Signal`。
2. 在发送方调用 `emit()`，在接收方 `connect()`。

4.6 新增一个聊天会话（session_id 隔离）

1. 确定唯一的 `session_id`（固定名称或 UUID）。
2. 在 `main.py` 中创建 `AgentWorker` 时传入 `configurable={"session_id": "voice-chat"}`，实现隔离。
3. 若需 UI 管理，可复用 `chat_session_changed` 信号或新增信号，连接后更新 `AgentController.configurable`。

🚪 5. 以「退出」为例说明完整扩展流程

下面回顾为"退出"菜单项新增功能的完整步骤，帮助理解可扩展机制。

❓ 5.1 原来的问题

原来用 `add_custom("menu_quit", "退出")`，类型为 `CUSTOM`，点击时只发送 `menu_item_triggered`，没有监听者，因此无反应。

✅ 5.2 改造为预定义类型 QUIT

1. **扩展枚举**：在 `MenuItemType` 中增加 `QUIT = auto()`。
2. **定义触发行为**：在 `MenuItem.on_triggered()` 中发射 `menu_quit_requested` 和 `application_quit_requested`。
3. **增加构建器方法**：在 `MenuBuilder` 中添加 `add_quit()`。
4. **使用新方法**：在 `LoverWindow` 中替换为 `.add_quit("退出")`。
5. **确认退出执行**：`WindowManager._setup_event_routing()` 中 `application_quit_requested.connect(self.quit)`，最终调用 `QApplication.quit()`。

⚠️ 5.3 为什么不直接 `sys.exit()`

- `sys.exit()` 立即终止进程，Qt 资源可能来不及释放。
- `QApplication.quit()` 正常退出事件循环，触发析构，更安全。
- 如需退出前保存配置，可连接 `application_quit_requested` 做预处理。

---

6. 开发约定

- **中文注释**：所有组件、窗口、暴露接口、关键逻辑必须写中文注释，特别是组件功能和暴露接口。
- **解耦**：窗口之间不直接调用，必须通过 EventBus。
- **预留接口**：未实现的功能用 `# TODO:` 标记，并暴露清晰的接口供后续填充。
- **目录规范**：
  - 窗口代码 → `windows/`
  - 可复用组件 → `widgets/`
  - 通用组件接口 → `widgets/base/`
- **Avatar 解耦**：`LoverWindow` 不直接依赖 Live2D，所有 Avatar 交互通过 `AvatarControllerProtocol`。

7. 快速参考：常用文件对应关系

需求	需要修改的文件
新增窗口	`windows/xxx_window.py`、`windows/window_manager.py`、`core/event_bus.py`
新增组件	`widgets/xxx/xxx.py`、在窗口 `setup_ui()` 中使用
新增右键菜单项	`widgets/menu/menu_item.py`、`widgets/menu/menu_builder.py`、`windows/lover_window.py`、`core/event_bus.py`
新增设置项类型	`widgets/settings/setting_item.py`
新增设置项实例	`windows/setting_window.py`
新增全局事件	`core/event_bus.py`
调整 Avatar 交互	`avatar/avatar_controller_protocol.py`、`avatar/controller.py`
迷你聊天（快速对话）	`windows/mini_chat_overlay.py`、`widgets/mini_chat/speech_bubble.py`、`core/event_bus.py`、`main.py`
会话管理（session_id 隔离）	`windows/chat_window.py`、`core/event_bus.py`、`main.py`
AgentWorker 思考信号路由	`main.py`（`AgentWorker` 参数与 `run()` 条件发射）
系统提示自动显示	`main.py`（`init_agent_controller` + `QTimer.singleShot`）
浮层 z-order 跟随	`windows/mini_chat_overlay.py`（`_on_target_moved` 中 `self.raise_()`）

需求	需要修改的文件
▶️ 运行真实应用	`main.py`
🍼 运行 UI 原型演示	`demos/desktop_ui_demo.py`

> 💡 **提示**：开发时请严格遵循事件总线通信模式，保持各模块独立可测试。后续任何扩展都可参照第 4 节步骤快速集成。